

TTP v2

Tarek Transport Protocol — implementação didática de um protocolo de transporte confiável sobre raw sockets IPv4, com comparação a TCP e UDP e captura de tráfego via Wireshark/Scapy.

- Python 3.10+
- Raw Sockets
- Sliding Window
- Handshake 3-way
- Retransmissão

Sumário

- 1. Visão geral do projeto
- 2. Arquitetura em camadas
- 3. O protocolo TTP em detalhes
 - a. Formato do cabeçalho e do pacote
 - b. Encapsulamento sobre IPv4 (raw sockets)
 - c. Checksum e pseudo-cabeçalho
 - d. Máquina de estados da conexão
 - e. Handshake de 3 vias (estabelecimento)
 - f. Números de sequência e reconhecimento (ACK)
 - g. Janela deslizante e controle de fluxo
 - h. Retransmissão por timeout
 - i. Recepção, buffer e dados fora de ordem
 - j. Encerramento gracioso (FIN / FIN-ACK)
- 4. Referência de funções e classes por arquivo
- 5. TCP e UDP: implementações de referência
- 6. TRP: um mini "Registro TLS" experimental
- 7. Captura de tráfego com Wireshark/Scapy
- 8. Como executar o projeto (passo a passo)
- 9. Requisitos, permissões e limitações conhecidas
- 10. Glossário

1. Visão geral do projeto

Este projeto é um laboratório didático de redes de computadores escrito em Python. Seu objetivo é permitir que o(a) leitor(a) compare, lado a lado, três formas de transportar dados entre um cliente e um servidor:

- **TCP** — o protocolo de transporte confiável e orientado a conexão do próprio sistema operacional, acessado via `socket.SOCK_STREAM` ;
- **UDP** — o protocolo de transporte não confiável e sem conexão do sistema operacional, acessado via `socket.SOCK_DGRAM` ;
- **TTP (Tarek Transport Protocol)** — um protocolo de transporte *confiável e orientado a conexão*, mas implementado inteiramente em espaço de usuário, em Python puro, sobre *raw sockets* IPv4. É o coração deste projeto e o foco principal desta documentação.

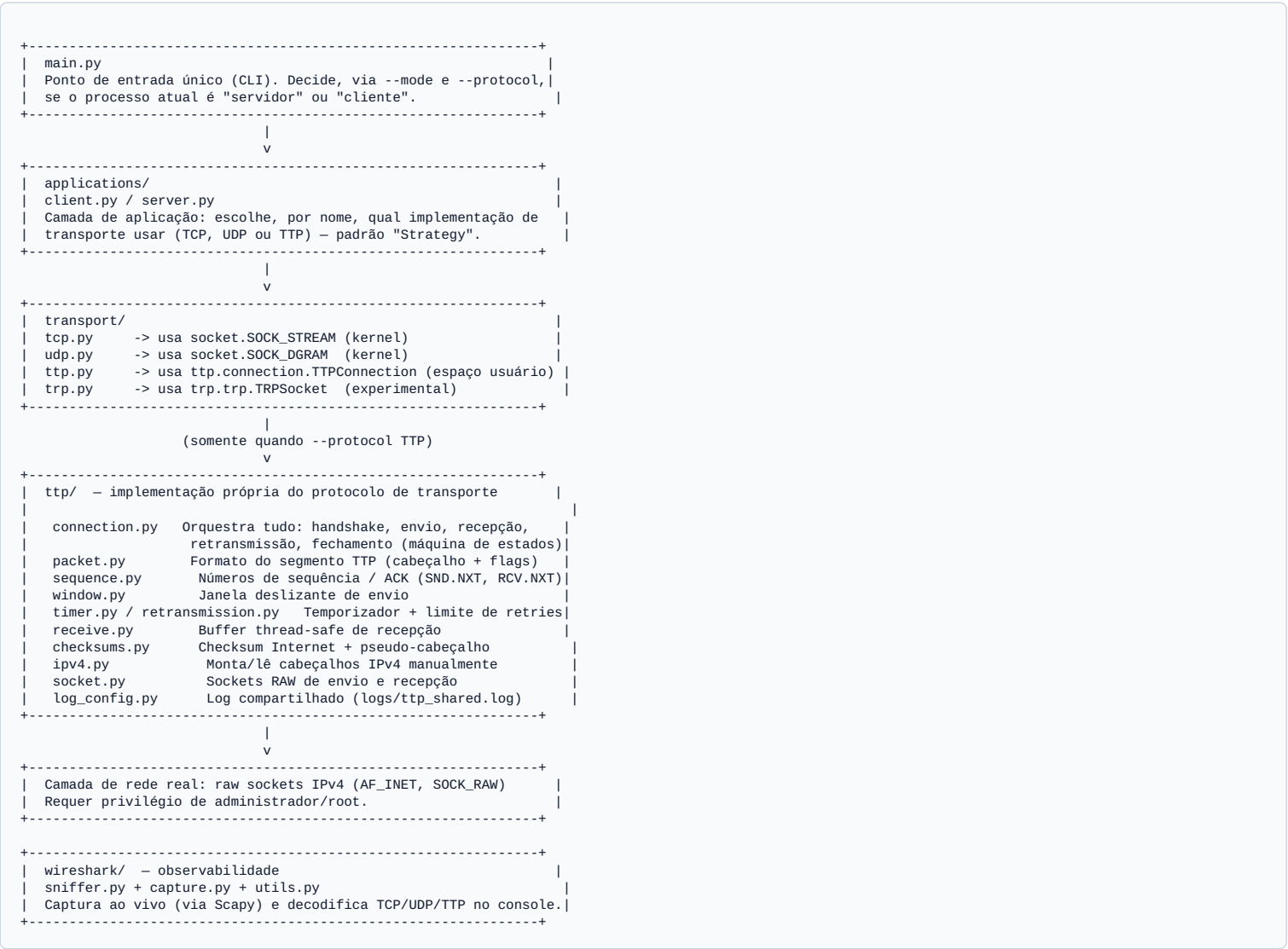
Além disso, o projeto inclui um quarto experimento, o **TRP (Tarek Record Protocol)**, uma camada de enquadramento de mensagens inspirada no TLS Record Protocol, e um **sniffer didático** baseado em Scapy que decodifica ao vivo o tráfego TCP, UDP e TTP gerado pela aplicação — funcionando como um "Wireshark simplificado", especializado neste projeto.

O TTP existe para responder, na prática, à pergunta: "*o que o TCP faz por baixo dos panos para parecer confiável, mesmo rodando sobre uma rede (IP) que não garante entrega nem ordem?*". A resposta, implementada aqui, envolve seis ingredientes centrais, cada um com seu próprio módulo dentro do pacote `ttp/`: números de sequência, reconhecimento cumulativo (ACK), janela deslizante, temporizadores de retransmissão, checksum de integridade e uma máquina de estados de conexão com handshakes de abertura e fechamento.

Por que "TTP" e não "TCP"? O nome evita confusão com o protocolo real e sinaliza que se trata de uma reimplementação simplificada, para fins de ensino — não um substituto de produção para o TCP.

2. Arquitetura em camadas

O projeto está organizado em camadas que se mapeiam quase diretamente ao modelo TCP/IP, cada uma isolada em seu próprio diretório:

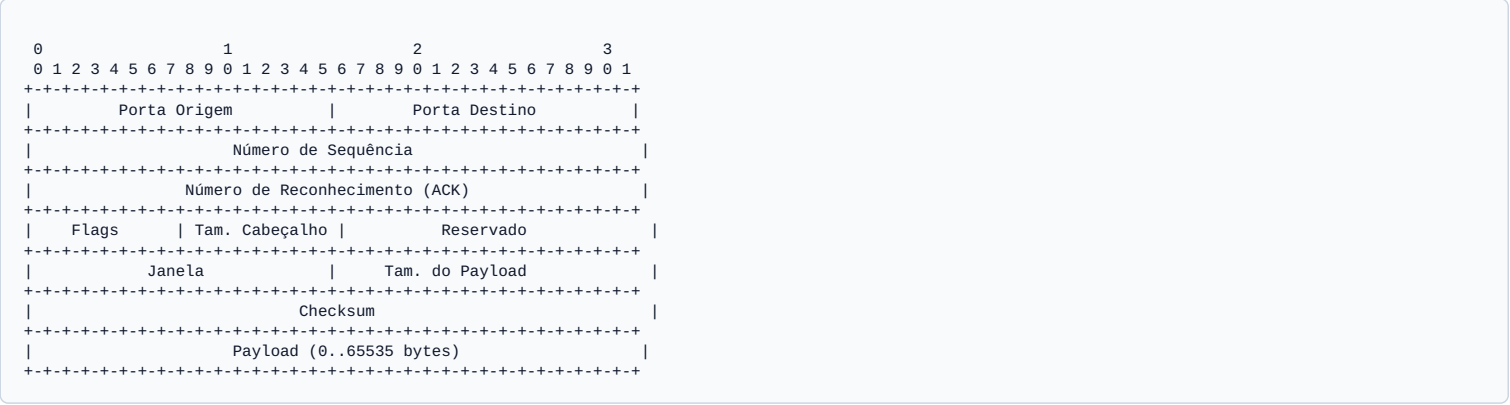


Essa separação em camadas permite trocar o transporte usado pela aplicação apenas mudando o argumento `--protocol` na linha de comando — a camada de aplicação (`applications/`) não precisa saber como cada transporte funciona por dentro.

3. O protocolo TTP em detalhes

3.a Formato do cabeçalho e do pacote

Cada segmento TTP é composto por um cabeçalho fixo de 24 bytes seguido de um payload de tamanho variável (até 65535 bytes). O layout é definido em `ttp/constants.py` (`HEADER_FORMAT = "!HHIIBBHHHI"`) e implementado em `ttp/packet.py` (classe `TTPPacket`):



Campo	Tamanho	Descrição
Porta Origem / Destino	2 + 2 bytes	Identificam a aplicação de origem e destino, exatamente como em TCP/UDP.
Número de Sequência	4 bytes	Posição do primeiro byte deste segmento no fluxo de dados enviado por este lado.
Número de ACK	4 bytes	Próximo byte que este lado espera receber do outro lado (reconhecimento cumulativo).
Flags	1 byte	Bits de controle: <code>SYN</code> , <code>ACK</code> , <code>FIN</code> , <code>RST</code> , <code>DATA</code> (ver <code>TTPFlags</code>).
Tamanho do cabeçalho	1 byte	Offset, em bytes, de onde começa o payload.
Reservado	2 bytes	Não utilizado; presente para alinhamento e futura extensão.
Janela	2 bytes	Tamanho da janela de recepção anunciada por este lado.
Tamanho do payload	2 bytes	Quantidade de bytes de dados que seguem o cabeçalho.
Checksum	4 bytes	Checksum Internet calculado sobre um pseudo-cabeçalho + o segmento inteiro.

Diferente do TCP real (que usa um checksum de 16 bits), este campo no TTP foi definido com 4 bytes por simplicidade de implementação em Python — o algoritmo de checksum utilizado, porém, é o mesmo "Internet checksum" de 16 bits (RFC 1071), apenas armazenado em um campo maior.

Flags (TTPFlags)

Flag	Valor	Significado
SYN	0x01	Sincroniza números de sequência — início do handshake.
ACK	0x02	O campo de reconhecimento é válido.
FIN	0x04	O emissor terminou de enviar dados — início do encerramento.
RST	0x08	Reset da conexão (definida, mas não utilizada ativamente na lógica atual).
DATA	0x10	O segmento carrega dados de aplicação.

Cada segmento também "consome" espaço de sequência conforme a propriedade `sequence_space`: um SYN ou um FIN consomem 1 número de sequência mesmo sem payload (assim como no TCP real), e um segmento DATA consome exatamente o número de bytes do seu payload.

3.b Encapsulamento sobre IPv4 (raw sockets)

Como o TTP não existe no kernel do sistema operacional, ele não pode ser transportado como um payload TCP ou UDP comum. Em vez disso, o projeto usa um **número de protocolo IP customizado (253)** — dentro da faixa oficialmente reservada pela IANA para "experimentação e testes" — e monta, byte a byte, seu próprio cabeçalho IPv4 (classe `IPv4Packet` em `ttp/ipv4.py`).

A classe `TTPSocket` (`ttp/socket.py`) abre **dois sockets RAW**:

- Um socket de **envio** (`IPPROTO_RAW` + `IP_HDRINCL=1`), que exige que o próprio programa forneça o datagrama IPv4 completo, cabeçalho incluso;
- Um socket de **recepção** aberto com o protocolo customizado (253), fazendo o kernel filtrar e entregar apenas datagramas IP marcados como TTP.

Atenção: sockets RAW exigem privilégios administrativos. No Linux/macOS é necessário rodar como `root` (ou com a capability `CAP_NET_RAW`); no Windows, é necessário executar como Administrador — e, na prática, o suporte a `SOCK_RAW` de envio de datagramas arbitrários é limitado fora do Linux, então recomenda-se WSL para rodar o modo TTP no Windows.

3.c Checksum e pseudo-cabeçalho

O módulo `http/checksums.py` implementa o clássico **checksum de Internet** (RFC 1071): soma todas as palavras de 16 bits do buffer em aritmética de complemento de um, propaga o "vai-um" de volta para os 16 bits baixos, e inverte o resultado bit a bit.

Assim como TCP e UDP, o TTP não calcula o checksum apenas sobre o segmento — ele inclui um **pseudo-cabeçalho** (IP de origem, IP de destino, protocolo e tamanho do segmento) que nunca é transmitido, mas participa do cálculo. Isso garante que o checksum também detecte segmentos entregues ao par errado de endereços IP, não apenas corrupção de bits.

3.d Máquina de estados da conexão

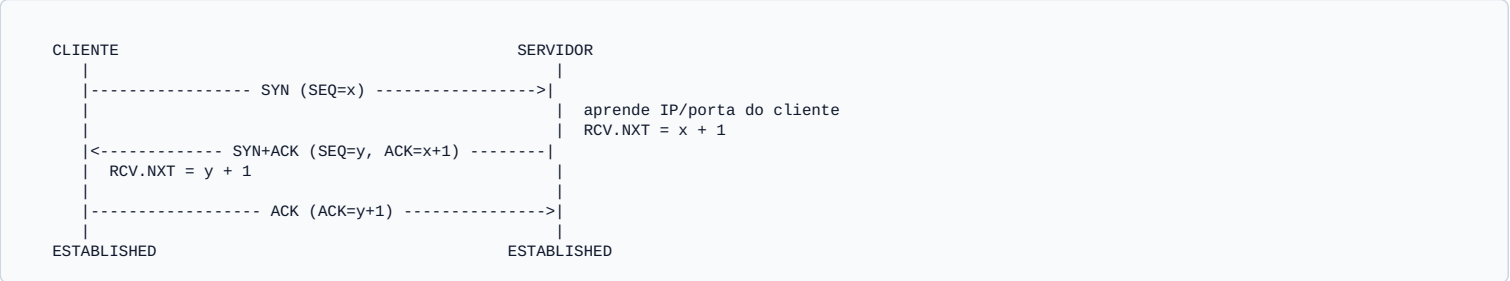
A classe `TTPState` (`http/packet.py`) define uma máquina de estados simplificada em relação às 11 fases do TCP real (RFC 793), cobrindo apenas o necessário para este projeto:



Estado	Significado
CLOSED	Nenhuma conexão ativa (estado inicial e final).
SYN_SENT	Lado cliente enviou SYN e aguarda SYN-ACK.
SYN_RECEIVED	Lado servidor recebeu SYN, enviou SYN-ACK e aguarda o ACK final.
ESTABLISHED	Handshake concluído; dados podem fluir nos dois sentidos.
FIN_WAIT	Este lado enviou (ou está processando) um FIN e aguarda confirmação.
CLOSING	Reservado para um encerramento simultâneo (não exercitado pela lógica atual de <code>close()</code>).

3.e Handshake de 3 vias (estabelecimento)

Implementado em `TTPConnection._client_handshake` e `TTPConnection._server_handshake`, o handshake segue exatamente a lógica do TCP:



Assim como no TCP, o número de sequência inicial (*ISN*) de cada lado é sorteado aleatoriamente (`SequenceSpace.__init__`) em vez de começar sempre em zero — isso evita que segmentos "fantasmas" de uma conexão anterior sejam confundidos com uma conexão nova.

3.f Números de sequência e reconhecimento (ACK)

A classe `SequenceSpace` (`http/sequence.py`) mantém três variáveis, com nomes inspirados diretamente na notação do RFC 793:

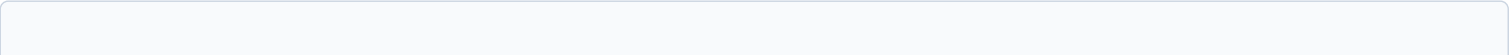
- `send_unacked` (SND.UNA) — o byte mais antigo enviado que ainda não foi confirmado;
- `send_next` (SND.NXT) — o próximo número de sequência a ser usado no envio;
- `recv_next` (RCV.NXT) — o próximo byte que este lado espera receber.

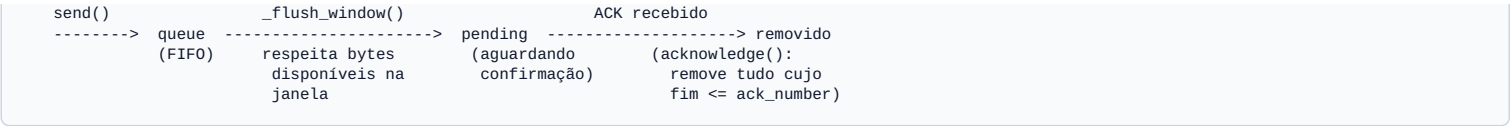
Ao receber um segmento, o método `receive()` classifica seu número de sequência em três categorias, refletidas no enum `ReceiveStatus`:

Status	Condição	Ação tomada
EXPECTED	<code>seq == recv_next</code>	Entrega imediata ao buffer de aplicação; avança <code>recv_next</code> .
DUPLICATE	<code>seq < recv_next</code>	Reenvia ACK (o ACK anterior provavelmente se perdeu); descarta o payload.
FUTURE	<code>seq > recv_next</code>	Armazena o pacote em <code>out_of_order</code> até que os bytes que faltam cheguem.

3.g Janela deslizante e controle de fluxo

A classe `SendWindow` (`http/window.py`) implementa o lado emissor da janela deslizante clássica de TCP: pacotes aguardam em uma fila (`queue`) até haver espaço na janela, momento em que são movidos para `pending` (enviados, aguardando ACK).





O tamanho da janela (`window_size`) é definido na criação da **TTPConnection** (por padrão, `DEFAULT_WINDOW_SIZE = 65535` , mas o projeto usa `4096` nos exemplos de `transport/ttp.py`). Um novo pacote só é efetivamente transmitido se `bytes_in_flight + tamanho_do_pacote <= size` — se a janela estiver cheia, o pacote aguarda na fila até que ACKs liberem espaço.

3.h Retransmissão por timeout

RetransmissionManager (`ttp/retransmission.py`) combina um temporizador simples (**TTPTimer**, baseado em `time.monotonic()`) com um contador de tentativas. O timeout é fixo (1 segundo, por padrão) e o número máximo de tentativas também (5, por padrão) — diferente do TCP real, que calcula um *RTO* adaptativo com base no RTT medido, o TTP usa valores fixos para simplicidade didática.

O temporizador é iniciado quando o primeiro pacote de uma rodada de envio se torna pendente (**TTPConnection._flush_window**) e interrompido assim que um ACK correspondente chega (**TTPConnection._process_ack**). O FIN de encerramento possui seu **próprio** gerenciador de retransmissão (`fin_retransmission`), independente do usado para dados.

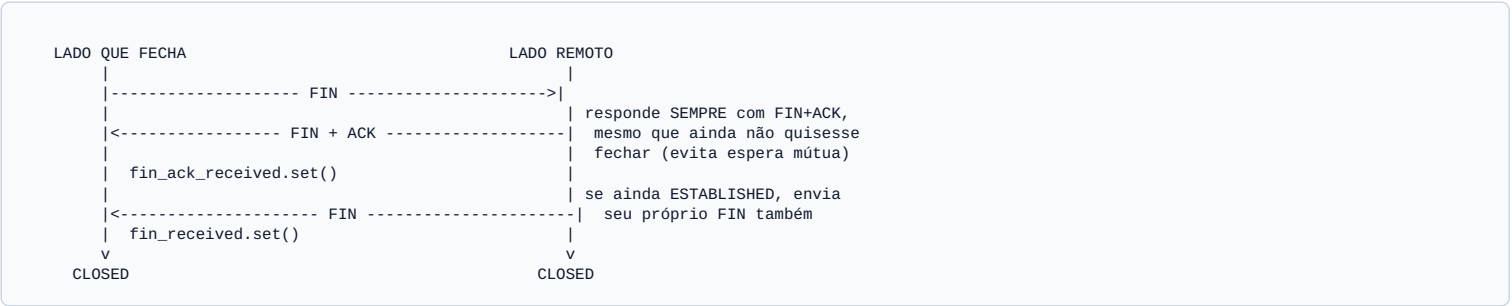
3.i Recepção, buffer e dados fora de ordem

Cada **TTPConnection** estabelecida roda uma **thread de recepção em segundo plano** (`_receive_loop`, iniciada por `_start_receiver`), que continuamente lê pacotes do socket RAW e os despacha conforme suas flags: ACK, FIN ou DATA.

Segmentos DATA que chegam fora de ordem (status `FUTURE`) são armazenados em um dicionário `out_of_order` , indexado pelo número de sequência. Assim que o "buraco" na sequência é preenchido por um segmento esperado, `_flush_out_of_order` libera em cadeia todos os segmentos armazenados que agora se tornaram contíguos, entregando-os na ordem correta ao **ReceiveBuffer**.

ReceiveBuffer (`ttp/receive.py`) é uma fila thread-safe simples: a thread de recepção empurra dados prontos (`push`) e a aplicação, ao chamar **TTPConnection.recv()**, retira o próximo item (`pop`), bloqueando até que haja algo disponível.

3.j Encerramento gracioso (FIN / FIN-ACK)



O método **TTPConnection.close()** envia um FIN e aguarda sua confirmação (com retransmissão, via `_wait_fin_ack`), depois aguarda até 5 segundos pelo FIN do lado remoto antes de liberar os sockets RAW. Isso evita perder dados que ainda não tenham sido confirmados e evita que os dois lados fiquem bloqueados esperando um FIN que nunca chegará (o outro lado sempre responde imediatamente com FIN+ACK ao receber um FIN).

4. Referência de funções e classes por arquivo

Esta seção documenta, arquivo a arquivo, as funções e classes mais relevantes do projeto.

ttp/packet.py

Item	Descrição
<code>TTPPacket.pack()</code>	Serializa o pacote (cabeçalho + payload) para bytes, prontos para envio.
<code>TTPPacket.unpack(data)</code>	Reconstrói um <code>TTPPacket</code> a partir de bytes recebidos, validando tamanhos.
<code>sequence_space</code>	Quantidade de números de sequência consumidos (payload + 1 se SYN/FIN).
<code>consumes_sequence</code>	Indica se o pacote avança o contador de sequência de envio.
<code>copy()</code>	Cria uma cópia independente, usada antes de recalculer o checksum.

ttp/ipv4.py

Item	Descrição
<code>IPv4Packet.build_header()</code>	Monta o cabeçalho IPv4 de 20 bytes, calculando seu checksum em duas passagens.
<code>IPv4Packet.pack()</code>	Retorna o datagrama IPv4 completo (cabeçalho + payload TTP).
<code>IPv4Packet.unpack(raw)</code>	Interpreta um datagrama IPv4 bruto recebido de um socket RAW.
<code>is_ttp()</code>	Verifica se o campo "Protocol" do datagrama corresponde ao TTP (253).

ttp/checksums.py

Item	Descrição
<code>checksum(data)</code>	Implementação genérica do checksum Internet (RFC 1071).
<code>build_ttp_checksum_data(...)</code>	Monta o pseudo-cabeçalho (IPs + protocolo + tamanho) concatenado ao segmento.
<code>calculate_ttp_checksum(...)</code>	Calcula o checksum final combinando pseudo-cabeçalho e segmento.
<code>validate_ttp_checksum(...)</code>	Recalcula o checksum de um pacote recebido e compara com o valor recebido.

ttp/socket.py

Item	Descrição
<code>TTPSocket.send_packet(...)</code>	Encapsula um <code>TTPPacket</code> em um datagrama IPv4 e o envia via socket RAW.
<code>TTPSocket.receive_packet()</code>	Bloqueia até chegar um datagrama IPv4 do protocolo TTP e o decodifica.
<code>_build_raw_packet(...)</code>	Calcula o checksum e monta o datagrama IPv4 final pronto para envio.
<code>_parse_raw_packet(...)</code>	Desencapsula o IPv4, valida o checksum e devolve o <code>TTPPacket</code> .

ttp/sequence.py

Item	Descrição
<code>SequenceSpace.can_send(size)</code>	Verifica se ainda há espaço na janela de envio para mais <code>size</code> bytes.
<code>SequenceSpace.receive(seq, amount)</code>	Classifica um segmento recebido como EXPECTED, DUPLICATE ou FUTURE.
<code>SequenceSpace.acknowledge(ack)</code>	Avança <code>send_unacked</code> com base em um ACK recebido (nunca retrocede).
<code>bytes_in_flight</code>	Bytes enviados e ainda não confirmados.

ttp/window.py

Item	Descrição
<code>SendWindow.enqueue(pkt)</code>	Adiciona um pacote à fila de envio.
<code>SendWindow.mark_sent()</code>	Move o pacote da frente da fila para o dicionário de pendentes.
<code>SendWindow.acknowledge(ack)</code>	Remove de <code>pending</code> todo pacote totalmente coberto pelo ACK recebido.
<code>bytes_available / can_send(size)</code>	Espaço livre na janela e verificação de admissão para novos envios.
<code>oldest()</code>	Retorna o pacote pendente mais antigo (referência do temporizador de retransmissão).

ttp/timer.py & ttp/retransmission.py

Item	Descrição
<code>TTPTimer.start() / .expired</code>	

	Cronômetro simples baseado em <code>time.monotonic()</code> .
<code>RetransmissionManager.start()</code> / <code>.restart()</code>	Inicia (zera tentativas) ou reinicia (incrementa tentativas) o ciclo de retransmissão.
<code>exhausted</code>	Indica que o número máximo de tentativas foi atingido — a conexão deve desistir.

`ttp/receive.py`

Item	Descrição
<code>ReceiveBuffer.push(data)</code>	Chamado pela thread de recepção ao aceitar dados em ordem.
<code>ReceiveBuffer.pop()</code>	Chamado pela aplicação (<code>recv()</code>); bloqueia até haver dado disponível.

`ttp/connection.py` – o orquestrador central

Método	Descrição
<code>connect()</code>	Executa o handshake ativo (cliente) e inicia a thread de recepção.
<code>accept()</code>	Executa o handshake passivo (servidor) e inicia a thread de recepção.
<code>send(data)</code>	Fragmenta <code>data</code> em segmentos, enfileira na janela e tenta transmitir.
<code>recv()</code>	Retira o próximo bloco de dados já reordenado do buffer de recepção.
<code>wait_for_acks(timeout)</code>	Bloqueia até a janela de envio esvaziar (tudo confirmado) ou estourar o timeout.
<code>close()</code>	Executa o encerramento gracioso via FIN/FIN-ACK, com retransmissão.
<code>_client_handshake()</code> / <code>_server_handshake()</code>	Implementam os dois lados do handshake de 3 vias.
<code>_process_ack(pkt)</code>	Atualiza sequência/janela e tenta liberar mais envios ao receber um ACK.
<code>_process_data(pkt)</code>	Classifica e trata um segmento DATA recebido (esperado, duplicado ou futuro).
<code>_process_fin(pkt)</code>	Responde a um FIN recebido com FIN+ACK e, se necessário, envia o próprio FIN.
<code>_flush_window()</code>	Transmite pacotes da fila enquanto houver espaço na janela.
<code>_flush_out_of_order()</code>	Libera, em cadeia, segmentos futuros armazenados que se tornaram contíguos.
<code>_receive_loop()</code>	Laço principal da thread de recepção em segundo plano.

5. TCP e UDP: implementações de referência

Os módulos `transport/tcp.py` e `transport/udp.py` usam diretamente o módulo `socket` da biblioteca padrão do Python, sem nenhuma lógica própria de confiabilidade — servem como "grupo de controle" para comparação com o TTP.

Aspecto	TCP (<code>transport/tcp.py</code>)	UDP (<code>transport/udp.py</code>)
Tipo de socket	<code>SOCK_STREAM</code>	<code>SOCK_DGRAM</code>
Conexão	Orientado a conexão (<code>create_connection</code> , <code>listen</code> / <code>accept</code>)	Sem conexão (<code>sendto</code> / <code>recvfrom</code> diretos)
Confiabilidade	Garantida pelo kernel	Nenhuma garantia
Uso neste projeto	Servidor lida com um cliente por vez, sem threads	Cada datagrama é tratado independentemente

6. TRP: um mini "Registro TLS" experimental

O pacote `trp/` contém uma implementação experimental do **Tarek Record Protocol (TRP)**, uma camada de enquadramento de mensagens inspirada no *TLS Record Protocol*. Cada registro TRP é formado por um cabeçalho fixo de 5 bytes (1 byte de tipo + 4 bytes de tamanho) seguido do payload:

+-----+-----+-----+-----+			
Tipo	Tamanho	Payload (N bytes)	
1 byte	4 bytes		
+-----+-----+-----+-----+			

O objetivo didático do TRP é demonstrar como um protocolo como o TLS resolve o problema de **enquadramento de mensagens sobre um fluxo de bytes** (TCP não preserva "limites de mensagem"): ao prefixar cada registro com seu tamanho exato, o receptor sempre sabe quantos bytes ler antes do próximo registro começar. O método utilitário `TRPSocket.recv_exact(sock, size)` ilustra esse cuidado, chamando `recv()` em loop até acumular exatamente o número de bytes esperado — já que TCP não garante que uma única chamada de `recv()` devolva a mensagem inteira de uma vez.

O TRP não está conectado ao seletor de protocolos de `applications/` (que só reconhece TCP, UDP e TTP); ele existe como módulo isolado para estudo do conceito de enquadramento de registros.

7. Captura de tráfego com Wireshark/Scapy

O diretório `wireshark/` contém um sniffer construído com a biblioteca **Scapy**, que funciona como um "Wireshark de bolso" dedicado a este projeto, decodificando ao vivo, no console, os três protocolos usados:

- `wireshark/capture.py` define uma classe Scapy customizada (**TTP(Packet)**) que ensina o Scapy a interpretar o cabeçalho TTP campo a campo — algo que o Scapy não sabe fazer nativamente, por não ser um protocolo padrão — e funções `capture_tcp`, `capture_udp` e `capture_ttp` que imprimem um resumo formatado de cada pacote (origem, destino, SEQ/ACK, flags, janela, checksum e payload).
- `wireshark/utils.py` fornece um `hexdump()` no estilo clássico offset/hex/ASCII, e um identificador heurístico de tipo de payload (texto UTF-8, JSON, requisição/resposta HTTP, binário).
- `wireshark/sniffer.py` é o script executável: registra o dissegador TTP customizado via `bind_layers(IP, TTP, proto=253)` e chama `scapy.sniff()` na interface de loopback (`lo`), filtrando apenas tráfego relevante a este projeto (porta 8443 para TCP/UDP, ou o protocolo 253 para TTP).

Para usar: abra um terminal com privilégios administrativos *antes* de rodar o cliente/servidor, e deixe o sniffer capturando:

```
# Linux/macOS
sudo python -m wireshark.sniffer

# Em seguida, em outros dois terminais, rode o servidor e o cliente
# normalmente (veja a seção 8).
```

8. Como executar o projeto (passo a passo)

8.1 Pré-requisitos

- Python 3.10 ou superior;
- Dependências: `cryptography` (para os certificados TLS) e `scapy` (apenas se for usar o sniffer);
- Para o modo **TTP**: privilégios de administrador/root (uso de raw sockets), e, no Windows, recomenda-se usar WSL.

```
pip install cryptography scapy
```

8.2 Rodando em modo TCP (referência)

```
# Terminal 1
python main.py --mode server --protocol TCP --port 8443

# Terminal 2
python main.py --mode client --protocol TCP --port 8443 --message "Olá via TCP!"
```

8.3 Rodando em modo UDP (referência)

```
# Terminal 1
python main.py --mode server --protocol UDP --port 8443

# Terminal 2
python main.py --mode client --protocol UDP --port 8443 --message "Olá via UDP!"
```

8.4 Rodando a rede em modo TTP (o foco deste documento)

O modo TTP usa *raw sockets* e por isso **precisa ser executado com privilégios de administrador/root**. No Linux/macOS, use `sudo`. No Windows, abra o terminal "Como Administrador" — porém a forma mais confiável de rodar sockets RAW arbitrários no Windows é através do WSL (Windows Subsystem for Linux), já que o suporte nativo do Windows a `SOCK_RAW` para protocolos customizados é restrito.

```
# Terminal 1 — sobe o "servidor" TTP, aguardando conexões na porta 8443
sudo python main.py --mode server --protocol TTP --host 127.0.0.1 --port 8443

# Terminal 2 — conecta como "cliente" TTP e envia uma mensagem
sudo python main.py --mode client --protocol TTP --host 127.0.0.1 --port 8443 \
  --message "Olá servidor via TTP!"
```

O que acontece, em ordem, ao rodar os dois comandos acima:

1. O servidor cria um **TTPConnection** "ouvinte" e chama `accept()`, bloqueando à espera de um SYN.
2. O cliente cria seu próprio **TTPConnection** e chama `connect()`, disparando o handshake de 3 vias (SYN → SYN-ACK → ACK).
3. Após o handshake, ambos os lados iniciam sua thread de recepção em segundo plano.
4. O cliente chama `send()` com a mensagem informada em `--message`, que é fragmentada, enfileirada na janela deslizante e transmitida.
5. O cliente aguarda a confirmação de tudo que enviou (`wait_for_acks`) antes de iniciar o encerramento.
6. O servidor chama `recv()`, recebe a mensagem já reordenada, e a imprime no console.
7. Ambos os lados chamam `close()`, executando o handshake de encerramento FIN/FIN-ACK.

Para acompanhar visualmente cada uma dessas etapas, execute o sniffer (seção 7) em um terceiro terminal antes de iniciar servidor e cliente, e observe também o arquivo de log `logs/ttp_shared.log`, que registra, com timestamp em microssegundos, cada evento interno do TTP (pacotes enviados, ACKs processados, estado da janela, retransmissões, etc.), tanto do lado `[Client]` quanto do lado `[Server]`, entrelaçados cronologicamente.

8.5 Gerando os certificados TLS (demo TLS separada)

```
pip install cryptography
python generate_certs.py
```

```
# Terminal 1
python server.py
```

```
# Terminal 2
python client.py
```

8.6 Atalhos no Windows (PowerShell)

```
.\run_generate_certs.ps1
.\run_server.ps1
```


9. Requisitos, permissões e limitações conhecidas

Item	Observação
Privilégios	O modo TTP e o sniffer Scapy exigem root/Administrador por dependerem de raw sockets / captura de pacotes.
Plataforma	Testado primariamente em ambiente Linux; no Windows, o comportamento de <code>SOCK_RAW</code> é mais restrito — recomenda-se WSL.
Concorrência	Os servidores TCP/UDP/TTP deste projeto atendem essencialmente uma conexão/mensagem por vez — não há pool de threads para múltiplos clientes simultâneos.
Congestionamento	O TTP implementa controle de fluxo (janela deslizante), mas não implementa controle de congestionamento (sem equivalentes a slow start, congestion avoidance, etc., presentes no TCP real).
RTO fixo	O timeout de retransmissão é um valor fixo (1s), diferente do RTO adaptativo (baseado em RTT medido) do TCP real.
Fragmentação IP	O módulo <code>ttp/ipv4.py</code> não implementa fragmentação/remontagem de datagramas IP — cada segmento TTP deve caber em um único datagrama.
Segurança	TTP não oferece nenhuma forma de criptografia ou autenticação — apenas checksum de integridade contra corrupção acidental, não contra adulteração maliciosa.

10. Glossário

Termo	Definição
ACK	Acknowledgment — reconhecimento de que dados foram recebidos com sucesso.
Checksum	Valor calculado a partir de um bloco de dados, usado para detectar erros de transmissão.
Handshake	Troca inicial de mensagens usada para estabelecer (ou encerrar) uma conexão.
ISN	Initial Sequence Number — número de sequência inicial escolhido no início de uma conexão.
Janela deslizante	Mecanismo de controle de fluxo que limita quantos bytes podem estar "em trânsito" sem confirmação.
Pseudo-cabeçalho	Estrutura auxiliar (nunca transmitida) incluída no cálculo do checksum para também proteger contra roteamento incorreto.
Raw socket	Socket que permite construir/ler pacotes de rede manualmente, abaixo do nível normalmente exposto pelo sistema operacional.
RTO	Retransmission Timeout — tempo que um emissor espera antes de retransmitir um pacote não confirmado.
Segmento	Unidade de dados de um protocolo de transporte (equivalente a "pacote" no contexto de TCP/TTP).
SYN	Synchronize — flag que inicia a sincronização de números de sequência no handshake.